

Как да напишем exam.cpp

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · vector, итератори, алгоритми

Какво ще направим

В упражнение 3 написахме динамичен масив на ръка: указател, брояч, капацитет, функция `grow`, деструктор, копиращ конструктор и оператор `=`. Около седемдесет реда, всеки от които може да се сгреша.

Стандартната библиотека предлага същото в един ред:

```
vector<CEXamRow> m_vecRows;
```

`vector` сам заделя и освобождава памет, сам расте, сам се копира правилно. Правилото на тримата отпада — няма какво да се управлява ръчно.

Ще напишем програма, която оценява студентски изпит: има **еталон** (условието с точките по части) и **решение** на студента. Програмата сравнява двете и изчислява резултата.

Клас	Какво пази
<code>CEXamRow</code>	Един ред от условието: име на частта, точки, съдържание.
<code>CStudent</code>	Име, <code>vector<CEXamRow></code> и сумарни точки.

План на работата

1. Нов проект и скелет
2. Класът `CEXamRow` — данни и конструктори (изискване I.1)
3. Операторите `<`, `==` и `<<` (изисквания I.2–I.4)
4. Статичната функция `addPoints` (изискване I.6)
5. Класът `CStudent` и векторът (изискване II.1)
6. Функцията `Output` и итерациите (изискване II.2)
7. Изпитването `examination` (изискване II.4)
8. Операторите на `CStudent` (изисквания II.5–II.6)
9. Главна функция и алгоритмите от STL (изискване III)

1 Нов проект и скелет

Нов **Empty Project**, например `Тема9_Exam`, файл `exam.cpp`.

```
#include <algorithm> // sort, find, count_if, max_element
#include <cstdio>     // getchar
```

```
#include <iomanip>    // setw, left, right
#include <iostream>
#include <numeric>    // accumulate
#include <string>
#include <vector>
using namespace std;
```

accumulate е в **<numeric>**, не в **<algorithm>**. Това е класическа загуба на време — грешката е C3861: 'accumulate': identifier not found, а всички останали алгоритми работят.

ПОМОЩНИТЕ ФУНКЦИИ ЗА КИРИЛИЦА

Както в упражнение 6, **setw** брои байтове, а кирилската буква е два. Пренесете двете функции:

```
static size_t utf8Length(const string& str)
{
    size_t nLen = 0;
    for (size_t i = 0; i < str.length(); i++)
        if ((static_cast<unsigned char>(str[i]) & 0xC0) != 0x80)
            nLen++;
    return nLen;
}

static string padRight(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return str + string(nWidth - nLen, ' ');
}
```

2 Класът CExamRow

```
class CExamRow
{
private:
    string m_strPart;        // име на частта, напр. "I."
    int    m_nPoints;        // точки
    string m_strContent;     // съдържание, напр. "Създаване_на_обекти"

public:
    // I.1 Три конструктора
    CExamRow();
    CExamRow(const string& strPart, int nPoints, const string& strContent);
    CExamRow(const CExamRow& row);

    // I.5 Четене на точките
    int getPoints() const;

    // Допълнителни акцесори – нужни при извеждането
    const string& getPart() const;
    const string& getContent() const;

    bool operator<(const CExamRow& row) const;    // I.2
    bool operator==(const CExamRow& row) const;   // I.3
```

```

        friend ostream& operator<<(ostream& toStream, const CExamRow& row);    // I.4

        static int addPoints(int nCurrent, const CExamRow& row);              // I.6
};

```

```

CExamRow::CExamRow()
    : m_strPart("няма"), m_nPoints(0), m_strContent("няма") {}

CExamRow::CExamRow(const string& strPart, int nPoints, const string& strContent)
    : m_strPart(strPart), m_nPoints(nPoints), m_strContent(strContent) {}

CExamRow::CExamRow(const CExamRow& row)
    : m_strPart(row.m_strPart), m_nPoints(row.m_nPoints), m_strContent(row.m_strContent) {}

```

Защо тук няма деструктор и оператор = ? Класът няма нито един указател — всичките му членове (string, int) се грижат за себе си. Генерираните от компилатора са напълно коректни. Това е втората половина на правилото на тримата от упражнение 3: *ако няма ресурс за управление, не се пише нито една от трите функции.*

Копирацията конструктор тук е написан само защото условието го изисква. Дори без него класът щеше да работи.

3 Операторите <, == и <<

```

// I.2 Сравнение за по-малко: CAMO по име на частта
bool CExamRow::operator<(const CExamRow& row) const
{
    return m_strPart < row.m_strPart;
}

// I.3 Равенство: по име на частта И съдържание
// Точките НЕ участват – същата задача, решена за други точки, е същата задача.
bool CExamRow::operator==(const CExamRow& row) const
{
    return m_strPart == row.m_strPart && m_strContent == row.m_strContent;
}

```

Тук < и == не си пасват — и това е нарочно. Два реда с еднакво име на частта, но различно съдържание не са равни по ==, а по < нито единият не е по-малък от другия.

Условието изисква точно това: подреждането е по части, а разпознаването на решена задача — по част и съдържание. Следствието е практически: такъв < **не бива** да се дава на set или map, защото те смятат за равни елементи, при които нито a < b, нито b < a — и биха изгубили редове. За sort и find обаче е напълно достатъчен.

ОПЕРАТОРЪТ ЗА ИЗВЕЖДАНЕ

```

ostream& operator<<(ostream& toStream, const CExamRow& row)
{
    toStream << padRight(row.m_strPart, 6)
        << setw(4) << right << row.m_nPoints << " т.  "

```

```

        << row.m_strContent;
    return toStream;
}

```

Същите правила като в упражнение 4: глобална приятелска функция, `ostream&` на връщане, `const` обект, без `endl`.

4 Статичната функция `addPoints`

```

// Сигнатурата (натрупано, елемент) -> натрупано е точно тази, която
// очаква алгоритъмът accumulate от <numeric>.
int CExamRow::addPoints(int nCurrent, const CExamRow& row)
{
    return nCurrent + row.m_nPoints;
}

```

КАКВО ЗНАЧИ `STATIC` В КЛАС

- 1 **Функцията принадлежи на класа, не на обект.** Вика се `CExamRow::addPoints(...)`, без да има нито един създаден обект.
- 2 **Няма `this`.** Затова не може да чете член-променливи направо — работи само с това, което ѝ е подадено. И затова не се пише `const` накрая: няма обект, който да бъде променен.
- 3 **`static` се пише само при обявяването в класа.** При дефиницията след класа думата не се повтаря — иначе C2724: 'static' should not be used on member functions defined at file scope.

Защо условието иска точно тази сигнатура. Формата „взема натрупаното дотук и следващия елемент, връща новото натрупано“ е стандартната форма на двоична операция за натрупване. Алгоритъмът `accumulate` приема точно такава функция:

```
int nTotal = accumulate(vecEtalon.begin(), vecEtalon.end(), 0, CExamRow::addPoints);
```

`accumulate` започва от `0` и за всеки елемент вика функцията, подавайки ѝ досегашния резултат. Името на функцията без скоби е нейният адрес — предава се като стойност, също като указател.

5 Класът `CStudent` и векторът

```

class CStudent
{
private:
    string          m_strName;      // име на студента
    vector<CExamRow> m_vecRows;     // вектор от обекти от клас CExamRow
    int             m_nTotal;      // сумарни точки от изпита

public:
    CStudent();
    CStudent(const string& strName, const vector<CExamRow>& vecRows); // II.1

    const string& getName() const; // II.3
}

```

```

const vector<CEXamRow>& getRows() const;
int getTotal() const;

ostream& Output(ostream& toStream) const;           // II.2
int examination(const vector<CEXamRow>& vecEtalon);   // II.4

friend ostream& operator<<(ostream& toStream, const CStudent& stud); // II.5
bool operator<(const CStudent& stud) const;         // II.6
};

```

```

CStudent::CStudent()
: m_strName("(няма име)", m_vecRows(), m_nTotal(0)) {}

CStudent::CStudent(const string& strName, const vector<CEXamRow>& vecRows)
: m_strName(strName), m_vecRows(vecRows), m_nTotal(0) {}

```

Този конструктор копира цял вектор с един ред. `m_vecRows(vecRows)` вика копиращия конструктор на `vector`, който заделва нова памет и копира всеки елемент — а всеки елемент се копира с копиращия конструктор на `CEXamRow`. Всичко това е дълбоко копиране и е безплатно за нас.

Сравнете със стъпка 5 от упражнение 3, където същото се пишеше ръчно.

Параметърът е `const vector<CEXamRow>&`, не `vector<CEXamRow>`. Без `&` векторът се копира *два пъти*: веднъж при предаването на параметъра и втори път в списъка за инициализация. При голям вектор това е чиста загуба.

ЧЕТЕНЕТО НА ПРОМЕНЛИВИТЕ

```

const string& CStudent::getName() const           { return m_strName; }
const vector<CEXamRow>& CStudent::getRows() const { return m_vecRows; }
int CStudent::getTotal() const                    { return m_nTotal; }

```

`getRows` връща `const` референция: извикващият вижда вектора, без да го копира и без да може да го промени.

6 Функцията `Output`

```

ostream& CStudent::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << m_vecRows.size() << " реда)" << endl;
    for (size_t i = 0; i < m_vecRows.size(); i++)
        toStream << "      " << m_vecRows[i] << endl;
    return toStream;
}

```

ТРИТЕ НАЧИНА ДА СЕ ОБХОДИ ВЕКТОР

Начин	Как изглежда	Кога
-------	--------------	------

по индекс	<code>for (size_t i = 0; i < v.size(); i++) ... v[i]</code>	когато индексът ви трябва; работи само за <code>vector</code> и <code>deque</code>
с итератор	<code>for (auto it = v.begin(); it != v.end(); ++it) ... *it</code>	работи за всеки контейнер, включително <code>list</code> , <code>set</code> и <code>map</code>
цикъл по обхват	<code>for (const CExamRow& row : v) ... row</code>	най-кратко, когато не ви трябва индекс (C++11)

Какво е итератор. Обект, който сочи към елемент от контейнера и се държи като указател: `*it` дава елемента, `++it` минава на следващия. `begin()` сочи първия, `end()` — мястото **след** последния.

Затова условието на цикъла е `it != v.end()`, а не `it < v.end()`: за `list` и `set` няма „по-малко“ между итератори, а `!=` работи навсякъде. Свиквайте с `!=` отсега.

`size()` връща `size_t`. Ако сравните с `int`, Visual Studio предупреждава C4267.

7 Изпитването `examination`

Това е същината на задачата: по подадения вектор-еталон се изчисляват точките на студента.

```
// За всеки ред на студента търсим същия ред в еталона (operator==).
// Ако го има – добавяме точките ОТ ЕТАЛОНА, не тези на студента.
int CStudent::examination(const vector<CExamRow>& vecEtalon)
{
    m_nTotal = 0;
    for (size_t i = 0; i < m_vecRows.size(); i++)
    {
        vector<CExamRow>::const_iterator it =
            find(vecEtalon.begin(), vecEtalon.end(), m_vecRows[i]);

        if (it != vecEtalon.end())
            m_nTotal = CExamRow::addPoints(m_nTotal, *it);    // I.6
    }
    return m_nTotal;
}
```

ТРИТЕ РЕШЕНИЯ В ТАЗИ ФУНКЦИЯ

- 1 **`find` използва вашия `operator==`.** Алгоритъмът не знае нищо за `CExamRow` — той просто сравнява с `==`. Затова `operator==` от стъпка 3 определя какво значи „студентът е решил тази задача“: съвпадение по част и съдържание.
- 2 **`find` връща `end()`, ако не намери.** Това е стандартният начин за „няма такъв“ в STL. Проверката `it != vecEtalon.end()` е задължителна — разадресирането на `end()` е грешка.
- 3 **Точките се вземат от `*it` — от еталона.** Не от `m_vecRows[i]`. Иначе студентът сам би си определял точките: в примера той е записал `0` точки на третата задача. Оценява еталонът.

Функцията и записва, и връща резултата. Условието иска точно това: „изчислява и записва в член променливата... (връща се като резултат)“. Затова тя не е `const` — променя `m_nTotal`. Преди извикването ѝ `getTotal()` връща 0.

8 Операторите на `CStudent`

```
// II.5 Извеждане: име и сумарни точки
ostream& operator<<(ostream& toStream, const CStudent& stud)
{
    toStream << padRight(stud.m_strName, 18) << setw(4) << right << stud.m_nTotal << " т.";
    return toStream;
}

// II.6 Сравнение по сумарни точки
bool CStudent::operator<(const CStudent& stud) const
{
    return m_nTotal < stud.m_nTotal;
}
```

Два оператора за извеждане в един клас. `Output` показва целия вектор — подробният вид. `operator<<` показва само име и точки — редът от класацията. Условието изисква и двете, защото служат за различни неща.

9 Главна функция и алгоритмите

9.1 Еталонът и решението на студента

```
vector<CEXamRow> vecEtalon;
vecEtalon.push_back(CEXamRow("I.", 10, "Създаване_на_клас_с_частни_членове"));
vecEtalon.push_back(CEXamRow("II.", 15, "Създаване_на_обекти_с_експлицитен_конструктор"));
vecEtalon.push_back(CEXamRow("III.", 20, "Предефиниране_на_оператор_за_извеждане"));
vecEtalon.push_back(CEXamRow("IV.", 25, "Работа_с_вектор_от_обекти"));
vecEtalon.push_back(CEXamRow("V.", 30, "Прилагане_на_алгоритъм_от_STL"));

CStudent examiner("ИЗПИТВАЩ (еталон)", vecEtalon);

// Студентът е решил част от задачите; една с различно съдържание.
vector<CEXamRow> vecStud;
vecStud.push_back(CEXamRow("I.", 10, "Създаване_на_клас_с_частни_членове"));
vecStud.push_back(CEXamRow("II.", 15, "Създаване_на_обекти_с_експлицитен_конструктор"));
vecStud.push_back(CEXamRow("III.", 0, "Предефиниране_на_оператор_за_въвеждане"));
vecStud.push_back(CEXamRow("V.", 30, "Прилагане_на_алгоритъм_от_STL"));

CStudent student("Иван Петров", vecStud);
```

Данните са нарочно подбрани, за да се провери всеки случай. Части I, II и V съвпадат напълно. Част III е капанът: името на частта съвпада, но съдържанието е „за_въвеждане“ вместо „за_извеждане“ — значи `operator==` дава лъжа и точки не се начисляват. Част IV изобщо липсва.

Очакван резултат: $10 + 15 + 30 = 55$ от възможни 100.

9.2 Извеждане и изпитване

```
examiner.Output(cout);
student.Output(cout);

int nResult = student.examination(vecEtalon);
cout << "  examination() върна: " << nResult << "  точки" << endl;
cout << "  чрез оператор <=:    " << student << endl;
```

9.3 accumulate със статичната функция

```
int nMaxPossible = accumulate(vecEtalon.begin(), vecEtalon.end(),
                              0, CExamRow::addPoints);
cout << "  максимално възможни точки (accumulate): " << nMaxPossible << endl;
cout << "  процент: " << (student.getTotal() * 100 / nMaxPossible) << "%" << endl;
```

Три аргумента плюс функцията: откъде, докъде, начална стойност, как се натрупва. Началната `0` определя и типа на резултата — ако напишете `0.0`, резултатът ще е `double`.

9.4 max_element и count_if

```
vector<CExamRow>::const_iterator itMax =
    max_element(vecEtalon.begin(), vecEtalon.end());
cout << "  последна част по азбучен ред (max_element): " << itMax->getPart() << endl;

int nHeavy = static_cast<int>(count_if(vecEtalon.begin(), vecEtalon.end(),
    [](const CExamRow& row) { return row.getPoints() >= 20; }));
cout << "  задачи с 20 или повече точки (count_if):    " << nHeavy << endl;
```

`max_element` ползва `operator<`, а той сравнява по име на частта. Затова резултатът е „V.“ — последната по азбучен ред, а не задачата с най-много точки. Съвпада случайно, защото частите са подредени и по двете. За „най-много точки“ трябва изричен сравнител:

```
max_element(vecEtalon.begin(), vecEtalon.end(),
    [](const CExamRow& a, const CExamRow& b) { return a.getPoints() <
    b.getPoints(); });
```

Поука: алгоритъмът винаги ползва `operator<` на типа, освен ако не му дадете друг. Проверявайте дали това е подредбата, която искате.

Какво е `[](const CExamRow& row) { ... }`. Това е **лямбда** — функция без име, написана на място. Квадратните скоби отбелязват началото ѝ; вътре в тях се изброява какво от околния код да е видимо (тук нищо). Удобна е точно за такива еднократни условия — иначе би трябвало да се напише отделна функция или функтор като `CLess` от упражнение 6.

9.5 `sort` — един алгоритъм, две подредби

```
vector<CEXamRow> vecSorted = vecEtalon;
sort(vecSorted.begin(), vecSorted.end());           // по ИМЕ НА ЧАСТТА

...

sort(vecStudents.begin(), vecStudents.end());       // по СУМАРНИ ТОЧКИ
```

Един и същи `sort`, два различни резултата — защото двата класа имат различен `operator<`. Алгоритъмът не съдържа никакво знание за данните.

`sort` работи върху `vector`, но не и върху `list`. Той иска произволен достъп — да може да скочи на произволен елемент. Списъкът предлага само движение стъпка по стъпка, затова има собствена функция `lst.sort()`. Това ще има значение в упражнение 11.

9.6 Обратно обхождане

```
for (vector<CStudent>::const_reverse_iterator it = vecStudents.rbegin();
     it != vecStudents.rend(); ++it)
    cout << " " << *it << endl;
```

За класация от най-добрия надолу не е нужно второ сортиране — достатъчно е да се обходи наобратно. `rbegin()` сочи последния елемент, `rend()` — мястото преди първия, а `++it` върви назад.

9.7 `[]` срещу `at()`

```
try
{
    // Първо работата, после извеждането — иначе редът излиза разкъсан.
    const string& strPart = vecEtalon.at(99).getPart();
    cout << " at(99)      = " << strPart << endl;
}
catch (const out_of_range& ex)
{
    // Текстът на съобщението е различен при различните компилатори.
    cout << " at(99)      -> хванато out_of_range: " << ex.what() << endl;
}
```

	<code>v[i]</code>	<code>v.at(i)</code>
Проверка на индекса	няма	има
При невалиден индекс	неопределено поведение — чете чужда памет	хвърля <code>out_of_range</code>
Скорост	малко по-бърз	една проверка повече

`v[99]` върху вектор с 5 елемента не дава грешка. Връща каквото е в паметта на това място. Програмата продължава със случайна стойност и се чупи по-късно, на друго място. Използвайте `at()` навсякъде, където индексът идва отвън.

Текстът на съобщението зависи от компилатора: Visual Studio казва `invalid vector subscript`, а други — само `vector`. Затова не пишете код, който разчита на съдържанието на `what()`.

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build.
<code>F5</code>	Start Debugging. В прозореца Watch напишете <code>vecEtalon</code> — Visual Studio показва вектора като списък от елементи, разгъващ се с триъгълника.

```
cl /EHsc /std:c++17 /utf-8 exam.cpp
exam.exe
```

В Debug режим програмите с контейнери са значително по-бавни. Visual Studio въвежда проверки на всяка операция с итератор. За измерване на скорост използвайте Release. За учебната задача разликата няма значение.

Кирилица под Windows

Файлът се записва като *Unicode (UTF-8 with signature)* — *Codepage 65001*; компилаторът получава `/utf-8`; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main`.

Защо съдържанието е с долни черти. Създаване_на_обекти вместо Създаване на обекти — така редът се чете с едно `>>`, ако утре данните дойдат от файл. Същото съображение като в упражнение 5.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 9: Контейнерни класове, алгоритми.
// CExamRow – един ред от условието на изпитна задача.
// CStudent – студент с вектор от такива редове и сумарни точки.

#include <algorithm> // sort, find, count_if, max_element
#include <cstdio>     // getchar
#include <iomanip>    // setw, left, right
#include <iostream>
#include <numeric>    // accumulate
#include <string>
#include <vector>
using namespace std;

// =====
// Помощно: подравняване при кирилица (setw брои байтове)
// =====
static size_t utf8Length(const string& str)
{
    size_t nLen = 0;
    for (size_t i = 0; i < str.length(); i++)
        if ((static_cast<unsigned char>(str[i]) & 0xC0) != 0x80)
            nLen++;
    return nLen;
}

static string padRight(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return str + string(nWidth - nLen, ' ');
}

// =====
// I. Клас CExamRow – един ред от условието
// =====
class CExamRow
{
// Скрита член променлива
private:
    string m_strPart;        // име на частта, напр. "I."
    int     m_nPoints;        // точки
    string  m_strContent;     // съдържание, напр. "Създаване_на_обекти"

public:
    // I.1 Три конструктора
    CExamRow();                // подразбиращ се
    CExamRow(const string& strPart, int nPoints, const string& strContent);
    CExamRow(const CExamRow& row); // копиращ

    // I.5 Четене на точките
    int getPoints() const;
```

```

// Допълнителни акцесори – нужни при извеждането
const string& getPart() const;
const string& getContent() const;

// I.2 Сравнение за по-малко ПО ИМЕ НА ЧАСТТА
bool operator<(const CExamRow& row) const;

// I.3 Сравнение за равенство ПО ИМЕ НА ЧАСТТА И СЪДЪРЖАНИЕ
bool operator==(const CExamRow& row) const;

// I.4 Извеждане в поток
friend ostream& operator<<(ostream& toStream, const CExamRow& row);

// I.6 Статична функция: текущи точки + обект -> нови точки
static int addPoints(int nCurrent, const CExamRow& row);
};

// ----- I.1 Конструктори -----
CExamRow::CExamRow()
    : m_strPart("(няма)", m_nPoints(0), m_strContent("(няма)") {}

CExamRow::CExamRow(const string& strPart, int nPoints, const string& strContent)
    : m_strPart(strPart), m_nPoints(nPoints), m_strContent(strContent) {}

CExamRow::CExamRow(const CExamRow& row)
    : m_strPart(row.m_strPart), m_nPoints(row.m_nPoints), m_strContent(row.m_strContent) {}

// ----- I.5 Четене -----
int CExamRow::getPoints() const          { return m_nPoints; }
const string& CExamRow::getPart() const  { return m_strPart; }
const string& CExamRow::getContent() const { return m_strContent; }

// ----- I.2 Сравнение за по-малко: CAMO по име на частта -----
bool CExamRow::operator<(const CExamRow& row) const
{
    return m_strPart < row.m_strPart;
}

// ----- I.3 Равенство: по име на частта И съдържание -----
// Точките НЕ участват – същата задача, решена за други точки, е същата задача.
bool CExamRow::operator==(const CExamRow& row) const
{
    return m_strPart == row.m_strPart && m_strContent == row.m_strContent;
}

// ----- I.4 Извеждане -----
ostream& operator<<(ostream& toStream, const CExamRow& row)
{
    toStream << padRight(row.m_strPart, 6)
              << setw(4) << right << row.m_nPoints << " т.  "
              << row.m_strContent;
    return toStream;
}

// ----- I.6 Статична функция за натрупване -----
// Сигнатурата (натрупано, елемент) -> натрупано е точно тази, която
// очаква алгоритъмът accumulate от <numeric>.
int CExamRow::addPoints(int nCurrent, const CExamRow& row)
{

```

```

        return nCurrent + row.m_nPoints;
    }

// =====
// II. Клас CStudent
// =====
class CStudent
{
private:
    string          m_strName;    // име на студента
    vector<CEXamRow> m_vecRows;    // вектор от обекти от клас CEXamRow
    int             m_nTotal;     // сумарни точки от изпита

public:
    // II.1 Конструктори
    CStudent();
    CStudent(const string& strName, const vector<CEXamRow>& vecRows);

    // II.3 Четене на променливите
    const string& getName() const;
    const vector<CEXamRow>& getRows() const;
    int getTotal() const;

    // II.2 Извеждане на вектора на указан поток
    ostream& Output(ostream& toStream) const;

    // II.4 Изпитване: по подадения вектор-еталон изчислява и записва точките
    int examination(const vector<CEXamRow>& vecEtalon);

    // II.5 Извеждане: име и сумарни точки
    friend ostream& operator<<(ostream& toStream, const CStudent& stud);

    // II.6 Сравнение "<" по сумарни точки
    bool operator<(const CStudent& stud) const;
};

// ----- II.1 Конструктори -----
CStudent::CStudent()
    : m_strName("(няма име)", m_vecRows(), m_nTotal(0)) {}

CStudent::CStudent(const string& strName, const vector<CEXamRow>& vecRows)
    : m_strName(strName), m_vecRows(vecRows), m_nTotal(0) {}

// ----- II.3 Четене -----
const string& CStudent::getName() const          { return m_strName; }
const vector<CEXamRow>& CStudent::getRows() const { return m_vecRows; }
int CStudent::getTotal() const                   { return m_nTotal; }

// ----- II.2 Извеждане на вектора -----
ostream& CStudent::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << m_vecRows.size() << " реда)" << endl;
    for (size_t i = 0; i < m_vecRows.size(); i++)
        toStream << "      " << m_vecRows[i] << endl;
    return toStream;
}

// ----- II.4 Изпитване -----
// За всеки ред на студента търсим същия ред в еталона (operator==).

```

```

// Ако го има – добавяме точките ОТ ЕТАЛОНА, не тези на студента.
int CStudent::examination(const vector<CEXamRow>& vecEtalon)
{
    m_nTotal = 0;
    for (size_t i = 0; i < m_vecRows.size(); i++)
    {
        vector<CEXamRow>::const_iterator it =
            find(vecEtalon.begin(), vecEtalon.end(), m_vecRows[i]);

        if (it != vecEtalon.end())
            m_nTotal = CExamRow::addPoints(m_nTotal, *it);    // I.6
    }
    return m_nTotal;
}

// ----- II.5 Извеждане: име и сумарни точки -----
ostream& operator<<(ostream& toStream, const CStudent& stud)
{
    toStream << padRight(stud.m_strName, 18) << setw(4) << right << stud.m_nTotal << " т.";
    return toStream;
}

// ----- II.6 Сравнение по сумарни точки -----
bool CStudent::operator<(const CStudent& stud) const
{
    return m_nTotal < stud.m_nTotal;
}

// =====
// III. Главна функция
// =====
int main()
{
    // ===== Еталонът: пълното условие на задачата =====
    vector<CEXamRow> vecEtalon;
    vecEtalon.push_back(CEXamRow("I.", 10, "Създаване_на_клас_с_частни_членове"));
    vecEtalon.push_back(CEXamRow("II.", 15, "Създаване_на_обекти_с_експлицитен_конструктор"));
    vecEtalon.push_back(CEXamRow("III.", 20, "Предефиниране_на_оператор_за_извеждане"));
    vecEtalon.push_back(CEXamRow("IV.", 25, "Работа_с_вектор_от_обекти"));
    vecEtalon.push_back(CEXamRow("V.", 30, "Прилагане_на_алгоритъм_от_STL"));

    // ===== III.1 Два обекта от клас CStudent =====
    CStudent examiner("ИЗПИТВАЩ (еталон)", vecEtalon);

    // Студентът е решил част от задачите; една с различно съдържание.
    vector<CEXamRow> vecStud;
    vecStud.push_back(CEXamRow("I.", 10, "Създаване_на_клас_с_частни_членове"));
    vecStud.push_back(CEXamRow("II.", 15, "Създаване_на_обекти_с_експлицитен_конструктор"));
    vecStud.push_back(CEXamRow("III.", 0, "Предефиниране_на_оператор_за_въвеждане"));
    vecStud.push_back(CEXamRow("V.", 30, "Прилагане_на_алгоритъм_от_STL"));

    CStudent student("Иван Петров", vecStud);

    cout << "===== III.1 Извеждане чрез Output =====" << endl;
    cout << "---- " ;
    examiner.Output(cout);
    cout << "---- ";
    student.Output(cout);
}

```

```

// ===== III.2 Изпитване и извеждане чрез << =====
cout << endl << "===== III.2 Изпитване =====" << endl;
int nResult = student.examination(vecEtalon);
cout << "  examination() върна: " << nResult << " точки" << endl;
cout << "  чрез оператор <<:    " << student << endl;

cout << "--- Как се получи резултатът ---" << endl;
const vector<CEXamRow>& vecRows = student.getRows();
for (size_t i = 0; i < vecRows.size(); i++)
{
    vector<CEXamRow>::const_iterator it =
        find(vecEtalon.begin(), vecEtalon.end(), vecRows[i]);
    cout << "  " << padRight(vecRows[i].getPart(), 6)
        << (it != vecEtalon.end()
            ? "намерен в еталона -> +" + to_string(it->getPoints()) + " т."
            : string("НЕ съвпада с еталона -> +0 т.))
        << endl;
}

// ===== Алгоритми върху вектора =====
cout << endl << "===== Алгоритми от STL =====" << endl;

// accumulate със статичната функция I.6
int nMaxPossible = accumulate(vecEtalon.begin(), vecEtalon.end(),
                               0, CEXamRow::addPoints);
cout << "  максимално възможни точки (accumulate): " << nMaxPossible << endl;
cout << "  получени от студента:                    " << student.getTotal() << endl;
cout << "  процент:                                    "
    << (student.getTotal() * 100 / nMaxPossible) << "%" << endl;

// max_element с operator<
vector<CEXamRow>::const_iterator itMax =
    max_element(vecEtalon.begin(), vecEtalon.end());
cout << "  последна част по азбучен ред (max_element): " << itMax->getPart() << endl;

// count_if с ламбда
int nHeavy = static_cast<int>(count_if(vecEtalon.begin(), vecEtalon.end(),
    [](const CEXamRow& row) { return row.getPoints() >= 20; }));
cout << "  задачи с 20 или повече точки (count_if):    " << nHeavy << endl;

// sort с operator< на CEXamRow – по ИМЕ НА ЧАСТТА
cout << "--- sort по име на частта (operator< на CEXamRow) ---" << endl;
vector<CEXamRow> vecSorted = vecEtalon;
sort(vecSorted.begin(), vecSorted.end());
for (size_t i = 0; i < vecSorted.size(); i++)
    cout << "  " << vecSorted[i] << endl;

// ===== Класация на студенти: sort с operator< на CStudent =====
cout << endl << "===== Класация (sort по сумарни точки) =====" << endl;

vector<CStudent> vecStudents;
vecStudents.push_back(student);

vector<CEXamRow> vecA;
vecA.push_back(vecEtalon[0]);
vecA.push_back(vecEtalon[3]);
CStudent studA("Елена Димитрова", vecA);
studA.examination(vecEtalon);
vecStudents.push_back(studA);

```

```

vector<CEXamRow> vecB = vecEtalon; // решил е всичко
CStudent studB("Георги Стоянов", vecB);
studB.examination(vecEtalon);
vecStudents.push_back(studB);

CStudent studC("Мария Иванова", vector<CEXamRow>()); // празен вектор
studC.examination(vecEtalon);
vecStudents.push_back(studC);

cout << "---- преди сортиране ----" << endl;
for (size_t i = 0; i < vecStudents.size(); i++)
    cout << " " << vecStudents[i] << endl;

sort(vecStudents.begin(), vecStudents.end()); // възходящо, operator<
cout << "---- след sort (възходящо по точки) ----" << endl;
for (size_t i = 0; i < vecStudents.size(); i++)
    cout << " " << vecStudents[i] << endl;

// Низходящо – чрез обратни итератори
cout << "---- низходящо (rbegin/rend) ----" << endl;
for (vector<CStudent>::const_reverse_iterator it = vecStudents.rbegin();
     it != vecStudents.rend(); ++it)
    cout << " " << *it << endl;

// ===== Основни операции с вектора =====
cout << endl << "===== Операции с vector =====" << endl;
cout << " size()      = " << vecEtalon.size() << endl;
cout << " empty()     = " << (vecEtalon.empty() ? "да" : "не") << endl;
cout << " front()    = " << vecEtalon.front().getPart() << endl;
cout << " back()     = " << vecEtalon.back().getPart() << endl;
cout << " at(2)      = " << vecEtalon.at(2).getPart() << endl;

// at() проверява индекса и хвърля – за разлика от []
try
{
    // Първо работата, после извеждането – иначе редът излиза разкъсан.
    const string& strPart = vecEtalon.at(99).getPart();
    cout << " at(99)      = " << strPart << endl;
}
catch (const out_of_range& ex)
{
    // Текстът на съобщението е различен при различните компилатори.
    cout << " at(99)      -> хванато out_of_range: " << ex.what() << endl;
}

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

Очакван изход

```

===== III.1 Извеждане чрез Output =====
--- ИЗПИТВАЩ (еталон) (5 реда)
I.      10 т. Създаване_на_клас_с_частни_членове

```

```

    II.    15 т. Създаване_на_обекти_с_експлицитен_конструктор
    III.   20 т. Предефиниране_на_оператор_за_извеждане
    IV.    25 т. Работа_с_вектор_от_обекти
    V.     30 т. Прилагане_на_алгоритъм_от_STL
--- Иван Петров (4 реда)
    I.     10 т. Създаване_на_клас_с_частни_членове
    II.    15 т. Създаване_на_обекти_с_експлицитен_конструктор
    III.    0 т. Предефиниране_на_оператор_за_въвеждане
    V.     30 т. Прилагане_на_алгоритъм_от_STL

```

==== III.2 Изпитване =====

```

examination() върна: 55 точки
чрез оператор <<: Иван Петров          55 т.

```

--- Как се получи резултатът ---

```

I.    намерен в еталона -> +10 т.
II.   намерен в еталона -> +15 т.
III.  НЕ съвпада с еталона -> +0 т.
V.    намерен в еталона -> +30 т.

```

==== Алгоритми от STL =====

```

максимално възможни точки (accumulate): 100
получени от студента:                    55
процент:                                55%
последна част по азбучен ред (max_element): V.
задачи с 20 или повече точки (count_if):  3
--- sort по име на частта (operator< на CExamRow) ---
I.     10 т. Създаване_на_клас_с_частни_членове
II.    15 т. Създаване_на_обекти_с_експлицитен_конструктор
III.   20 т. Предефиниране_на_оператор_за_извеждане
IV.    25 т. Работа_с_вектор_от_обекти
V.     30 т. Прилагане_на_алгоритъм_от_STL

```

==== Класация (sort по сумарни точки) =====

--- преди сортиране ---

```

Иван Петров      55 т.
Елена Димитрова 35 т.
Георги Стоянов  100 т.
Мария Иванова   0 т.

```

--- след sort (възходящо по точки) ---

```

Мария Иванова   0 т.
Елена Димитрова 35 т.
Иван Петров     55 т.
Георги Стоянов  100 т.

```

--- низходящо (rbegin/rend) ---

```

Георги Стоянов  100 т.
Иван Петров     55 т.
Елена Димитрова 35 т.
Мария Иванова   0 т.

```

==== Операции с vector =====

```

size()      = 5
empty()     = не
front()     = I.
back()      = V.
at(2)       = III.
at(99)      -> хванато out_of_range: vector

```

Чести грешки

Код	Съобщение	Причина и решение
C3861	'accumulate': identifier not found	Липсва <code>#include <numeric></code> . Той не е част от <code><algorithm></code> .
C2784	could not deduce template argument	Смесени итератори от различни контейнери, или <code>begin()</code> на един и <code>end()</code> на друг.
C2676	binary '==': 'CEXamRow' does not define this operator	<code>find</code> изисква <code>operator==</code> . Проверете дали е <code>const</code> член.
C2724	'static' should not be used on member functions defined at file scope	Думата <code>static</code> е повторена при дефиницията след класа. Пише се само в класа.
C4267	conversion from 'size_t' to 'int'	Резултатът на <code>size()</code> или <code>count_if</code> се присвоява на <code>int</code> . Използвайте <code>size_t</code> или изрично <code>static_cast<int></code> .
C2662	cannot convert 'this' pointer ... to 'CEXamRow &'	Извиква се неконстантна функция върху елемент от <code>const</code> вектор. Маркирайте акцесорите с <code>const</code> .
—	<code>examination</code> винаги връща 0	<code>operator==</code> не съвпада — най-често заради разлика в съдържанието. Изведете двата низа, за да ги сравните.
—	Срив при разадресиране	Използва се <code>*it</code> без проверка <code>it != v.end()</code> .
—	<code>sort</code> подрежда неочаквано	<code>operator<</code> сравнява по друг член, отколкото очаквате. Тук <code>CEXamRow</code> се подрежда по име на частта, не по точки.
—	Срив след <code>push_back</code> в цикъл с итератори	Добавянето към <code>vector</code> може да премести данните и да обезсили всички итератори. Не добавяйте, докато обхождате.

Обобщение: какво дава `vector`

Упражнение 3 — ръчно	Тук — с <code>vector</code>
<code>double* m_pdGrades; size_t m_nCount, m_nCapacity;</code>	<code>vector<CEXamRow> m_vecRows;</code>
функция <code>grow()</code> с ново заделяне и пренасяне	<code>push_back</code>
деструктор с <code>delete[]</code>	няма нужда
копиращ конструктор с дълбоко копиране	няма нужда
оператор <code>=</code> със самоприсвояване и освобождаване	няма нужда
ръчна проверка на индекса	<code>at()</code>

Упражнение 3 не беше загубено време — без него не е ясно *какво* прави `vector` вътре в себе си и защо копирането му не е безплатно. Но в реален код ръчното управление на памет се избягва точно по тази причина.

Какво следва

Упражнение 10 Още алгоритми: `sort`, `unique`, `remove` — търсене и премахване на дублирани записи.

Упражнение 11 `list` — контейнер без произволен достъп, със собствен `sort`.

Упражнения 12–13 `map` и `set` — асоциативни контейнери, където подредбата е част от устройството.
